

Executive Summary

The Autonomous Data Journalism Agent is a multi-agent AI system that turns raw entertainment-industry datasets into readable, data-driven news articles. It explores whether AI can handle the repetitive parts of data analysis—loading data, examining columns, calculating statistics, finding useful patterns, and reporting the results—without requiring a person to guide every step.

The system accepts either an uploaded CSV file or a Kaggle dataset reference. Although it was designed around music and film data, the workflow can also handle other well-structured tabular datasets. Rather than sending the full dataset to a language model, the project keeps it inside an isolated E2B environment. This allows Python to perform the mathematical and statistical work and makes it possible to analyze datasets with hundreds of thousands, or even more than one million, rows.

LangGraph coordinates five stages: ingestion, planning, analysis, article writing, and evaluation. Ingestion identifies columns, data types, row counts, file sizes, and basic statistics while preparing memory-conscious settings for large files. A Planner Agent uses that schema to propose dataset-specific investigation questions, with a default plan retained as a fallback. The Analyst Agent then writes a Python script using pandas, NumPy, and SciPy. Its cleaning rules adapt to the available columns and domain, so film-specific checks are not applied to music data. The script produces rankings, correlations, and time-based comparisons where the data supports them.

The Journalist Agent turns those results into a short news-style article. It is instructed not to introduce figures, names, or relationships that are missing from the analysis output. The Evaluator Agent then checks the draft. Programmatic checks look for ranking and correlation-direction problems, while a second language-model pass checks whether the wider claims and formatted numbers are supported by the results.

When the evaluator finds a problem, it sends specific feedback back through the workflow so the analysis and article can be revised. The system allows up to three attempts. If the article still cannot be verified, it is returned with a visible unverified warning rather than being presented as fully reliable. Gemini is the primary language-model provider, with Qwen available as a fallback if the primary request fails.

A Gradio interface lets users upload data, provide a Kaggle reference, follow the pipeline through a live log, and read the final article. The application is prepared for deployment on Hugging Face Spaces. Overall, the project demonstrates how agentic AI, sandboxed Python execution, and iterative fact-checking can work together in one continuous pipeline. Its main idea is straightforward: language models handle planning and communication, while Python remains responsible for the calculations that support the final story.

FIGURE 1

Application interface

Autonomous Data Journalism Agent

Upload a CSV of entertainment industry data, or give a Kaggle dataset ref to download it straight into the sandbox. The agent performs EDA, drafts an article, and verifies every numerical claim.

CSV dataset

spotify-2023.csv

103.8 KB

...or Kaggle dataset ref (owner/dataset-name)
owner/dataset-name

Run pipeline

Live agent log

Starting pipeline...
CSV size: 0.1 MB
Creating E2B sandbox...
E2B sandbox ready. Uploading CSV to E2B (0.1 MB)...
Upload complete. Building LangGraph workflow...
Running graph with dataset...

► ingest completed at +1.9s

► planner completed at +3.2s
planned investigations (chosen from the schema):
1. Clean first: public datasets contain junk/placeholder rows. Before any ranking or correlation, drop rows with missing, zero, or implausible sentinel values in t
2. Calculate the correlation coefficient between 'bpm' and 'danceability_%' to determine if faster-tempo tracks consistently score higher in danceability.
3. Group by 'artist(s)_name' and calculate the sum of 'streams' to rank the top 10 most-streamed artists in the dataset.
4. Compare the average 'energy_%' and 'acousticness_%' across different 'mode' (Major vs. Minor) categories to identify which musical mode dominates high-energy tr
5. Rank the top 10 tracks by the sum of 'in_spotify_charts', 'in_apple_charts', and 'in_deezer_charts' to identify the most cross-platform successful songs.
6. Calculate the average 'danceability_%', 'energy_%', and 'valence_%' for tracks grouped by 'released_year' to identify trends in popular music characteristics ov

MUSIC INDUSTRY DATA ANALYSIS REVEALS STREAMING TRENDS AND CROSS-PLATFORM PERFORMANCE

A comprehensive analysis of 952 tracks reveals distinct patterns in modern music consumption, highlighting the dominance of specific artists and the technical characteristics defining popular songs. The data indicates a slight negative correlation of -0.15 between a track's beats per minute and its danceability rating.

The Weeknd leads all artists in total streams, accumulating 14,185,555,287 streams. Taylor Swift follows as the second-highest-streamed artist with 14,053,658,300, while Ed Sheeran ranks third with 13,908,947,204. Other top-performing artists by total streams include Harry Styles, Bad Bunny, and Olivia Rodrigo, all of whom have surpassed 7 billion streams.

When examining cross-platform success, measured by total appearances on charts across three major services, "Seven (feat. Latto) (Explicit Ver.)" by Latto and Jung Kook holds the top position with 420 chart appearances. "Flowers" by Miley Cyrus ranks second with 388, followed by "As It Was" by Harry Styles with 374.

The analysis also highlights differences in musical composition based on mode. Songs recorded in a minor key exhibit higher average energy levels at 65.34% compared to 63.49% for major key tracks. Conversely, major key tracks feature higher average acousticness at 28.48%, compared to 25.17% for those in a minor key.

Historical data trends show fluctuating averages for danceability, energy, and valence over the past century. In 2023, the average danceability reached 70.02%, while average energy and valence were recorded at 68.22% and 55.08%, respectively.

The data underscores the significant reach of top-tier artists in the current streaming landscape.

The Gradio interface showing dataset input, live pipeline progress, and the generated article.

Practitioner's Notes

What We Built

We wanted to test whether AI could handle the path from an entertainment dataset to a checked news article without manual guidance. We kept numerical work away from the language models: they plan, generate code, interpret, and write, while pandas, NumPy, and SciPy perform the calculations inside a sandbox.

Why We Chose These Tools

We chose LangGraph because we needed shared state and a correction loop, not just a sequence of prompts. It carries the schema, investigation plan, scripts, outputs, draft, feedback, and revision count between five stages: ingestion, planning, analysis, journalism, and evaluation.

Our model choice was strongly influenced by being students. Gemini offers a free API key, so it was a practical primary provider for development and demonstrations. We also had access to a free Qwen key provided by the teacher of another elective course. Instead of leaving that resource unused, we configured Qwen as a fallback. If a Gemini request fails, the same request can be retried through Qwen's OpenAI-compatible endpoint.

We selected E2B because it felt modern and simple to integrate. It gave us an isolated environment where model-generated Python could run without executing directly on the application server. A new sandbox is created for each pipeline run and killed afterward. Uploaded CSV files are copied into it, while Kaggle datasets can be downloaded directly inside it. This was especially useful for large datasets because the main application does not need to download and re-upload the complete file.

Gradio provided file input, a Kaggle reference field, progress logs, and article display without a separate frontend. Hugging Face Spaces gave us a simple deployment target and supplies the pinned Gradio runtime.

FIGURE 2

Model routing

```

class FallbackChatModel:
    def __init__(self, primary: ChatModel, fallback: ChatModel) -> None:
        self.primary = primary
        self.fallback = fallback

    def invoke(self, messages: Any, *args: Any, **kwargs: Any) -> Any:
        try:
            return self.primary.invoke(messages, *args, **kwargs)
        except Exception as primary_error:
            try:
                return self.fallback.invoke(messages, *args, **kwargs)
            except Exception as fallback_error:
                raise RuntimeError(
                    "Gemini primary and Qwen fallback LLM calls both failed. "
                    f"Gemini error: {primary_error!r}. "
                    f"Qwen error: {fallback_error!r}."
                ) from fallback_error

def content_to_text(content: Any) -> str:
    if isinstance(content, str):
        return content
    if content is None:
        return ""
    if isinstance(content, list):
        parts = []
        for item in content:
            if isinstance(item, str):
                parts.append(item)
            elif isinstance(item, dict):
                if "text" in item:
                    parts.append(str(item["text"]))
                elif "content" in item:
                    parts.append(content_to_text(item["content"]))
                else:
                    parts.append(str(item))
            else:
                parts.append(str(item))
        return "\n".join(part for part in parts if part)
    return str(content)

def get_llm(
    temperature: float = 0.2,
    google_factory: Callable[..., ChatModel] | None = None,
    qwen_factory: Callable[..., ChatModel] | None = None,
) -> ChatModel:
    google_factory = google_factory or _google_factory
    qwen_factory = qwen_factory or _qwen_factory
    timeout = _float_env(
        "LLM_REQUEST_TIMEOUT_SECONDS",
        DEFAULT_LLM_REQUEST_TIMEOUT_SECONDS,
    )
    max_retries = _int_env("LLM_MAX_RETRIES", DEFAULT_LLM_MAX_RETRIES)

    try:
        primary = google_factory(
            model=os.getenv("GEMINI_MODEL", DEFAULT_GEMINI_MODEL),
            api_key=_required_env("GEMINI_API_KEY"),
            temperature=temperature,
            timeout=timeout,
            max_retries=max_retries,
        )
    except Exception:
        return qwen_factory(
            model=os.getenv("QWEN_MODEL", DEFAULT_QWEN_MODEL),

```

Gemini primary routing with Qwen fallback and response normalization.

FIGURE 3

Sandbox execution

```
def _parse_kaggle_manifest(
    f"Kaggle download for '{dataset_ref}' produced no manifest."
    f"stdout tail: {stdout[-400:]}!r} | stderr tail: {stderr[-400:]}!r}"
)
csv_files = payload["csv_files"]
if not csv_files:
    raise RuntimeError(f"No CSV files found in Kaggle dataset '{dataset_ref}'")
return csv_files

class Sandbox:
    def __init__(self) -> None:
        self._sbx = E2BSandbox.create(timeout=SANDBOX_TIMEOUT_SECONDS)
        self._dataset_remote_path: str | None = None
        self._context = None

    def upload(self, local_path: str, remote_name: str = "data.csv") -> str:
        with open(local_path, "rb") as f:
            self._sbx.files.write(f"/home/user/{remote_name}", f.read())
        self._dataset_remote_path = f"/home/user/{remote_name}"
        return self._dataset_remote_path

    def set_dataset_path(self, remote_path: str) -> str:
        self._dataset_remote_path = remote_path
        return self._dataset_remote_path

    def download_kaggle(self, dataset_ref: str) -> list[dict]:
        envs = {
            name: value
            for name in ("KAGGLE_USERNAME", "KAGGLE_KEY")
            if (value := os.environ.get(name))
        }
        self._sbx.commands.run("pip install -q kagglehub", timeout=300, envs=envs)

        code = "ref = " + repr(dataset_ref) + "\n" + KAGGLE_DOWNLOAD_CODE
        exec_result = self._sbx.run_code(code, envs=envs)
        return _parse_kaggle_manifest(
            stdout="\n".join(exec_result.logs.stdout),
            stderr="\n".join(exec_result.logs.stderr),
            error=exec_result.error,
            dataset_ref=dataset_ref,
        )

    def _fresh_context(self):
        if self._context is None:
            self._context = self._sbx.create_code_context()
        else:
            self._sbx.restart_code_context(self._context)
        return self._context

    def run(self, code: str) -> dict:
        context = self._fresh_context()
        exec_result = self._sbx.run_code(
            code, context=context, timeout=RUN_TIMEOUT_SECONDS
        )
        stdout = "\n".join(exec_result.logs.stdout)
        stderr = "\n".join(exec_result.logs.stderr)
        if exec_result.error:
            stderr += f"\n{exec_result.error.name}: {exec_result.error.value}"
        return {"stdout": stdout, "stderr": stderr}

    @property
    def dataset_path(self) -> str:
        if not self._dataset_remote_path:
            raise RuntimeError("No dataset uploaded")
        return self._dataset_remote_path
```

E2B dataset transfer, Kaggle download, code-context reset, and isolated execution.

How the Pipeline Works

Ingestion samples and profiles every CSV, records its schema and row count, and prepares memory-conscious `read_csv` settings. Long free-text columns can be excluded, while useful columns and secondary tables remain available.

The Planner uses the table names, columns, data types, and row counts to propose dataset-specific questions. The original EDA plan remains as a fallback if planning fails or returns too few ideas. Planning happens once; evaluator revisions return directly to the Analyst.

The Analyst generates one Python script from the Planner's questions. Cleaning follows the available columns and domain: film rules apply only to film-like schemas, while music data uses suitable ranges for streams, duration, and audio features. Results are printed as labelled raw numbers. The Journalist turns them into a short article without adding unsupported facts.

The Evaluator combines programmatic ranking and correlation-direction checks with an LLM review. Raw numeric matching is non-blocking because dates, ranks, rounding, and formatted numbers produced too many false positives. Wider value and context checks rely on the semantic evaluator—a practical compromise, not a perfect guarantee.

Failed evaluations return specific feedback to the next analysis attempt. The graph allows three attempts before returning the latest draft with an `[unverified]` warning. This prevents an endless loop while keeping failure visible.

FIGURE 4

Workflow orchestration

```
def build_graph(sandbox: Sandbox):
    workflow = StateGraph(AgentState)
    workflow.add_node("ingest", partial(ingest_node, sandbox=sandbox))
    workflow.add_node("planner", planner_node)
    workflow.add_node("analyst", partial(analyst_node, sandbox=sandbox))
    workflow.add_node("journalist", journalist_node)
    workflow.add_node("evaluator", evaluator_node)

    workflow.set_entry_point("ingest")
    workflow.add_edge("ingest", "planner")
    workflow.add_edge("planner", "analyst")
    workflow.add_edge("analyst", "journalist")
    workflow.add_edge("journalist", "evaluator")
    workflow.add_conditional_edges(
        "evaluator",
        evaluator_route,
        {"analyst": "analyst", "end": END},
    )
    return workflow.compile()

def initial_state(dataset_path: str) -> AgentState:
    return {
        "dataset_path": dataset_path,
        "schema": {},
        "analysis_plan": [],
        "analysis_results": [],
        "article_draft": "",
        "evaluation_errors": [],
        "revision_count": 0,
        "final_article": "",
    }

def run(dataset_path: str) -> str:
    load_dotenv()
    sandbox = Sandbox()
    try:
        graph = build_graph(sandbox)
        final = graph.invoke(initial_state(dataset_path), config={"recursion_limit": 25})
        return final["final_article"]
    finally:
        sandbox.close()

if __name__ == "__main__":
    if len(sys.argv) < 2:
        print("Usage: python -m src.graph <path-to-csv>")
        sys.exit(1)
    article = run(sys.argv[1])
    print("\n=== FINAL ARTICLE ===\n")
    print(article)
```

The five-stage LangGraph workflow and evaluator correction route.

FIGURE 5

Dataset ingestion

```
def ingest_node(state: AgentState, sandbox: Sandbox) -> dict:
    dataset_path = state["dataset_path"]
    if dataset_path.startswith(KAGGLE_PREFIX):
        dataset_ref = dataset_path[len(KAGGLE_PREFIX):]
        csv_files = sandbox.download_kaggle(dataset_ref)
        paths = [f["path"] for f in csv_files]
    elif dataset_path.startswith("/home/user/"):
        paths = [dataset_path]
    else:
        paths = [sandbox.upload(dataset_path)]

    code = "paths = " + repr(paths) + "\n" + INGEST_CODE
    result = sandbox.run(code)
    if result["stderr"]:
        raise RuntimeError(f"Ingest failed: {result['stderr']}")
    schema = json.loads(result["stdout"].strip().splitlines()[-1])

    primary = _primary_path(schema["files"])
    schema["primary_path"] = primary
    sandbox.set_dataset_path(primary)
    return {"schema": schema, "analysis_plan": _initial_plan(schema)}

def _primary_path(files: list[dict]) -> str:
    return max(files, key=lambda f: f.get("size_bytes", 0))["path"]

def _initial_plan(schema: dict) -> list[str]:
    files = schema["files"]
    all_cols = sorted({c for f in files for c in f["columns"]})
    table_summary = ", ".join(f"{f['name']} ({len(f['columns'])} cols)" for f in files)
    return [
        "Clean first: public datasets contain junk/placeholder rows. Before any "
        "ranking or correlation, drop rows with missing, zero, or implausible "
        "sentinel values in the metric being analyzed; where a vote/popularity "
        "count exists, restrict to sufficiently-rated entries; prefer released "
        "titles. Print how many rows remain after filtering.",
        f"Across the available tables [{table_summary}], identify the top 10 "
        f"entries ranked by the most prominent count/streams/revenue column "
        f"among {all_cols} – using the cleaned data.",
        "Compute pairwise correlations between numeric columns and surface the "
        "3 strongest (by |r|). If two tables share a key column, join them first "
        "to surface cross-table relationships.",
        "Identify any temporal column (year, release_date) and describe how key "
        "metrics shift across it.",
    ]
```

CSV ingestion, primary-table selection, and the fallback investigation plan.

FIGURE 6

Schema-driven planning

```
def _format_schema(files: list[dict]) -> str:
    blocks = []
    for f in files:
        cols = f.get("analysis_columns") or f.get("columns", [])
        dtypes = f.get("dtypes", {})
        col_lines = ", ".join(f"{c} ({dtypes.get(c, '?')})" for c in cols)
        blocks.append(
            f"- {f.get('name', f.get('path', 'table'))} "
            f"({f.get('n_rows', '?')} rows)\n  columns: {col_lines}"
        )
    return "\n".join(blocks)

def _parse_ideas(text: str) -> list[str]:
    ideas: list[str] = []
    match = re.search(r"\[.*\]", text, re.DOTALL)
    if match:
        try:
            data = json.loads(match.group(0))
        except (ValueError, TypeError):
            data = None
        if isinstance(data, list):
            ideas = [str(x).strip() for x in data if str(x).strip()]

    if not ideas:
        for line in text.splitlines():
            cleaned = re.sub(r"^\s*(?:[-*]|\d+[.])\s*", "", line).strip()
            if len(cleaned) >= 10:
                ideas.append(cleaned)

    seen: set[str] = set()
    unique: list[str] = []
    for idea in ideas:
        key = idea.lower()
        if key not in seen:
            seen.add(key)
            unique.append(idea)
    return unique[:MAX_IDEAS]

def planner_node(state: AgentState) -> dict:
    schema = state.get("schema") or {}
    files = schema.get("files") or []
    if not files:
        return {}

    fallback_plan = state.get("analysis_plan") or []
    cleaning_step = fallback_plan[0] if fallback_plan else None

    user = USER_TEMPLATE.format(schema=_format_schema(files))
    try:
        llm = get_llm(temperature=0.4)
        response = llm.invoke(
            [SystemMessage(content=SYSTEM), HumanMessage(content=user)]
        )
        ideas = _parse_ideas(content_to_text(response.content))
    except Exception:
        return {}

    if len(ideas) < MIN_IDEAS:
        return {}

    plan = ([cleaning_step] if cleaning_step else []) + ideas
    return {"analysis_plan": plan}
```

The Planner formats schema information and generates dataset-specific investigation questions.

FIGURE 7

Sandboxed analysis

```
def _extract_code(text: str) -> str:
    m = re.search(r"```(?:python)?\s*\n(?:.*?)```", text, re.DOTALL)
    return m.group(1).strip() if m else text.strip()

def _format_datasets(files: list[dict]) -> str:
    blocks = []
    for f in files:
        size_mb = f.get("size_bytes", 0) // (1024 * 1024)
        analysis_cols = f.get("analysis_columns") or f["columns"]
        dtypes = f.get("dtypes", {})
        analysis_dtypes = {c: dtypes.get(c) for c in analysis_cols}
        excluded = f.get("excluded_columns", [])
        block = (
            f"- {f.get('name', f['path'])} "
            f"({f.get('n_rows', '?')} rows, ~{size_mb} MB)\n"
            f"  path: {f['path']}\n"
            f"  analysis columns/dtypes: {analysis_dtypes}\n"
            f"  Read CSV settings – read with pd.read_csv(path, **read_csv_settings):\n"
            f"    read_csv_settings = {f.get('read_csv_kwargs', {})}"
        )
        if excluded:
            block += f"\n  (excluded heavy free-text columns: {excluded})"
        blocks.append(block)
    return "\n".join(blocks)

def _build_revision_notes(state: AgentState) -> str:
    parts = []
    runs = state.get("analysis_results", [])
    last = runs[-1] if runs else None
    if last and last.get("stderr", "").strip():
        parts.append(
            "YOUR PREVIOUS SCRIPT RAISED AN ERROR. Fix it so it runs cleanly:\n"
            + last["stderr"][:2000]
        )
    if last and not last.get("stdout", "").strip():
        parts.append(
            "YOUR PREVIOUS SCRIPT PRINTED NOTHING. A large CSV may have exhausted "
            "memory – read only needed columns / use chunksize, and make sure every "
            "result is printed with print()."
        )
    if state.get("evaluation_errors"):
        parts.append(
            "PREVIOUS ARTICLE FAILED VERIFICATION:\n"
            + "\n".join(f"- {e}" for e in state["evaluation_errors"])
            + "\nProvide additional statistics that resolve these issues."
        )
    return "\n\n".join(parts)

def analyst_node(state: AgentState, sandbox: Sandbox) -> dict:
    llm = get_llm(temperature=0.2)
    revision_notes = _build_revision_notes(state)
    user = USER_TEMPLATE.format(
        datasets=_format_datasets(state["schema"]["files"]),
        tasks="\n".join(f"- {t}" for t in state["analysis_plan"]),
        revision_notes=revision_notes,
    )
    response = llm.invoke([SystemMessage(content=SYSTEM), HumanMessage(content=user)])
    code = _extract_code(content_to_text(response.content))
    result = sandbox.run(code)
    run = {"script": code, "stdout": result["stdout"], "stderr": result["stderr"]}
    return {"analysis_results": state.get("analysis_results", []) + [run]}
```

The Analyst prepares revision context, generates Python, and records stdout and stderr.

FIGURE 8

Article drafting

```
def journalist_node(state: AgentState) -> dict:
    results = state["analysis_results"]
    if not any(r["stdout"].strip() for r in results):
        return {
            "article_draft": "INSUFFICIENT DATA: the analysis produced no output, "
            "so there are no verified findings to report."
        }
    llm = get_llm(temperature=0.4)
    analyses = "\n\n---\n\n".join(
        f"Script:\n{r['script']}\n\nStdout:\n{r['stdout']}"
        for r in state["analysis_results"]
    )
    revision_notes = ""
    if state.get("evaluation_errors"):
        revision_notes = (
            "PREVIOUS DRAFT HAD ERRORS:\n"
            + "\n".join(f"-- {e}" for e in state["evaluation_errors"])
            + "\nRewrite to remove these errors. Use only verified figures."
        )
    user = USER_TEMPLATE.format(analyses=analyses, revision_notes=revision_notes)
    response = llm.invoke([SystemMessage(content=SYSTEM), HumanMessage(content=user)])
    return {"article_draft": content_to_text(response.content).strip()}
```

The Journalist grounds a concise article in accumulated analysis output.

FIGURE 9

Semantic evaluation

```
def parse_llm_verdict(text: str) -> list[str]:
    match = re.search(r"\{.*\}", text, re.DOTALL)
    if not match:
        return []
    try:
        verdict = json.loads(match.group(0))
    except (ValueError, TypeError):
        return []
    if verdict.get("supported") is True:
        return []
    issues = verdict.get("issues") or []
    return [str(issue) for issue in issues if str(issue).strip()]

def verify_semantic(article: str, stdouts: list[str], llm=None) -> list[str]:
    if not article.strip() or not any(s.strip() for s in stdouts):
        return []
    llm = llm or get_llm(temperature=0.0)
    user = SEMANTIC_USER.format(article=article, stdout="\n".join(stdouts))
    try:
        response = llm.invoke(
            [SystemMessage(content=SEMANTIC_SYSTEM), HumanMessage(content=user)]
        )
    except Exception:
        return []
    return parse_llm_verdict(content_to_text(response.content))

_NUMBER_RE = re.compile(r"-?\d[\d,]*(?:\.\d+)?")
_POS_CORR_RE = re.compile(
    r"(positive(?:ly)?\s+(?:correlat|associat|relat))",
    re.IGNORECASE,
)
_NEG_CORR_RE = re.compile(
    r"(negative(?:ly)?\s+(?:correlat|associat|relat)|inverse(?:ly)?\s+(?:correlat|relat))",
    re.IGNORECASE,
)
_SUPERLATIVE_RE = re.compile(
    r"(?P<entity>[A-Z] [\w\.\&' -]{1,40}?)\s+is\s+the\s+most\s+(?P<metric>[a-z]+)",
    re.IGNORECASE,
)

def _normalize_num(token: str) -> str:
    return token.replace(",", "", "")
```

The semantic verifier parses the LLM verdict and checks contextual support.

FIGURE 10

Value and sign checks

```
def verify_values(article: str, stdouts: list[str]) -> list[str]:
    errors: list[str] = []
    joined = "\n".join(stdouts)
    stdout_nums = {_normalize_num(t) for t in _NUMBER_RE.findall(joined)}
    for token in _NUMBER_RE.findall(article):
        norm = _normalize_num(token)
        if len(norm) < 2:
            continue
        if norm in stdout_nums:
            continue
        try:
            val = float(norm)
        except ValueError:
            continue
        tolerance = max(abs(val) * 0.01, 0.01)
        if not any(
            abs(val - float(s)) <= tolerance
            for s in stdout_nums
            if _is_floatable(s)
        ):
            errors.append(f"Value '{token}' not found in stdout (no match within 1% tolerance)")
    return errors

def _is_floatable(s: str) -> bool:
    try:
        float(s)
        return True
    except ValueError:
        return False

def verify_signs(article: str, stdouts: list[str]) -> list[str]:
    errors: list[str] = []
    joined = "\n".join(stdouts)
    pos_in_article = bool(_POS_CORR_RE.search(article))
    neg_in_article = bool(_NEG_CORR_RE.search(article))

    corr_matches = re.findall(r"corr[^\s]*=\s*(-?\d*\.\d+)", joined, re.IGNORECASE)
    if not corr_matches:
        return errors
    signs = {"neg" if float(v) < 0 else "pos" for v in corr_matches}

    if pos_in_article and "pos" not in signs:
        errors.append("Article claims positive correlation but stdout shows none.")
    if neg_in_article and "neg" not in signs:
        errors.append("Article claims negative correlation but stdout shows none.")
    return errors
```

Programmatic extraction of numeric claims and correlation direction.

FIGURE 11

Rank verification and circuit breaker

```

def verify_ranks(article: str, stdouts: list[str]) -> list[str]:
    errors: list[str] = []
    joined = "\n".join(stdouts)
    matches = list(_SUPERLATIVE_RE.finditer(article))
    if not matches:
        return errors
    list_pattern = re.compile(r"\s*(([^\s]+)\s+)"
    first_items = list_pattern.findall(joined)
    if not first_items:
        return errors
    top_names = []
    for item in first_items:
        parts = [p.strip().strip('"') for p in item.split(",")]
        if parts:
            top_names.append(parts[0].lower())
    for m in matches:
        entity = m.group("entity").strip().lower()
        if entity not in top_names:
            errors.append(
                f"Article claims '{m.group('entity')}' is the most {m.group('metric')}, "
                f"but stdout's top entries are: {top_names}"
            )
    return errors

def select_blocking_errors(
    value_errors: list[str],
    sign_errors: list[str],
    rank_errors: list[str],
    semantic_errors: list[str],
) -> list[str]:
    blocking: list[str] = []
    for err in [*sign_errors, *rank_errors, *semantic_errors]:
        if err not in blocking:
            blocking.append(err)
    return blocking

def evaluator_node(state: AgentState, llm=None) -> dict:
    stdouts = [r["stdout"] for r in state["analysis_results"]]
    article = state["article_draft"]
    if not any(s.strip() for s in stdouts):
        errors = [
            "The analysis produced no output (empty stdout), so no claim in the "
            "article can be grounded in a computed statistic. Re-run the analysis "
            "so it prints real results before drafting."
        ]
    else:
        errors = select_blocking_errors(
            value_errors=verify_values(article, stdouts),
            sign_errors=verify_signs(article, stdouts),
            rank_errors=verify_ranks(article, stdouts),
            semantic_errors=verify_semantic(article, stdouts, llm=llm),
        )
    revision_count = state.get("revision_count", 0)
    if not errors:
        return {"evaluation_errors": [], "final_article": article}
    if revision_count + 1 >= MAX_REVISIONS:
        tagged = article + "\n\n[unverified] " + " ".join(errors)
        return {
            "evaluation_errors": errors,
            "revision_count": revision_count + 1,
            "final_article": tagged,
        }
    return {
        "evaluation_errors": errors,

```

Ranking checks, blocking-error selection, revision counting, and final routing.

Running the Project

The application normally needs GEMINI_API_KEY, QWEN_API_KEY, and E2B API access. It loads values from the environment or a local `.env` file; model names, Qwen's endpoint, timeouts, and retries can also be overridden. Kaggle credentials are optional where authentication is not required.

Hugging Face Spaces launches the root `app.py` with server-side rendering disabled. Local use requires the declared packages and Gradio runtime. The graph can also run through `python -m src.graph <path-to-csv>`.

Lessons and Limitations

Our strongest lesson was that language and numerical work should stay separate. LangGraph made the feedback loop understandable, and E2B made generated-code execution manageable. We also learned that fact-checking becomes difficult when natural writing introduces rounding, currencies, dates, and different ways of expressing the same value.

Although the system is complete and operational, there are areas we would improve. Planner responses rely mainly on prompt instructions rather than strict column validation, sampled descriptive statistics are stored but not passed to the Planner or Analyst, and outputs from earlier revisions accumulate. The repository also lacks an automated test suite and reproducible example datasets.

Our next improvements would be strict schema validation for planned questions, structured analysis results instead of free-form stdout, deterministic links between article claims and statistics, and separate routes for analysis errors and writing-only revisions. These changes would make verification stricter without creating unnecessary recomputation or false alarms.